Prompt Syntax (PS) — Specification Draft v0.2

Status: Working draft — design decisions marked ✓ (settled), ❖ (proposed, awaiting confirmation), **?** (open question) **Editors:** [author], with drafting assistance **Last updated:** 2026-07-19

> **Changelog v0.1 → v0.2:** major revision incorporating external platform-operator review (2026-07-19). Added: authority/specificity precedence split with non-escalation rule (§10); four-layer architecture (§4); trace visibility tiers (§12); pinned canonical identifiers (§6.2); capability negotiation (§7); parameter normalization records (§7.2); standardized failure codes (§8.2); budget construct replacing generic `limit()` (§9.1); fallback authorization rule (§9.2); determinism taxonomy (§14); mandatory island highlighting for interactive implementations (§13). Deferred: multi-model span execution (→ orchestration extension). Reframed: portability as *portable intent + transparent resolution, not portable execution*.

1. Abstract

Prompt Syntax (PS) is a vendor-neutral language for expressing **authorized prompt-execution intent**, together with a conformance protocol describing how that intent was resolved and fulfilled. Prompts remain natural language with embedded *syntax islands* — directives that give users **routing control** (deterministic, *authority-bounded* selection of models, tools, skills, and context) and **routing transparency** (an inspectable record of assembly, resolution, and fulfillment: the **Prompt Trace**). PS defines a grammar, resolution semantics, an authority model, and a conformance contract; it does not define presentation UX, and it promises portable *expression of intent* — never portable execution.

2. Terminology

Term	Definition
Directive	A PS construct embedded in a prompt that requests assembly, routing, or execution behavior.
Syntax island	A directive occurrence inside otherwise-plain natural language.
Entity	A referenceable resource: model, agent, plugin/connector, tool, skill, file, context source.
Canonical identifier	The immutable, versioned, vendor-qualified form of an entity reference (§6.2).
Resolver	The defined procedure mapping an entity reference to a canonical identifier.
Authority level	The governance tier a configuration value originates from (§10.1).
Capability envelope	The set of permissions and resources granted by all higher authority levels; directives operate only within it.
Fulfillment policy	The directive's contract for unsatisfiable requirements: `strict` / `partial` / `best-effort` (§8).
Fill report	The per-requirement requested → normalized → applied record, with reason codes (§8.2).
Budget	A venue-enforced execution bound with defined measurement points (§9.1).
Fallback chain	An ordered list of routing steps tried on fill failure; the only contingency construct in PS (§9.2).
Canonical form	The fully explicit representation every PS prompt compiles to (§11).
Prompt Trace	The tiered, machine-readable artifact recording assembly, resolution, and fulfillment (§12).
Trace tier	A defined visibility level of the trace: user / developer / operator / oversight (§12.2).
Routing control	The property that directives *guarantee* what runs — within the capability envelope — rather than the model inferring intent.
Routing transparency	The property that the user can inspect, at their tier, what will run (pre-execution) and what ran (post-execution).

3. Design principles ✓

1. **Graceful degradation.** A prompt with zero directives is valid PS; every directive is optional prose-compatible.
2. **Familiarity borrowing.** Every sigil reuses a convention with an existing large user base (@-mentions, /commands, XML tags, frontmatter).

3. **Progressive disclosure.** Casual form → parameterized form → strict form, all compiling to one canonical representation; the equivalence is a property of the grammar.
4. **Content safety.** Pasted code/HTML/XML must never accidentally become directives; only reserved namespaces are live.
5. **Contract, not presentation.** The spec mandates information availability (the Prompt Trace); presentation is implementation-defined — with one exception (§13, island highlighting).
6. **Minimal reserved surface.** One live tag namespace (`ps`), two sigils (`@`, `/`), one fence (`---ps`). The reserved set never grows across versions.
7. **One reference grammar.** The `@` entity-reference syntax is the sole routing syntax, reused verbatim at every scope.
8. **Non-escalation.** ✓ *(v0.2)* Prompt content can *narrow* the capability envelope; it can never expand it. No directive grants access, permission, or budget that higher authority has not already granted.
9. **Intent portability, not execution portability.** ✓ *(v0.2)* PS guarantees that intent is expressed portably and resolution is reported transparently; it never guarantees that execution is identical across venues.

4. Layered architecture ✓ *(v0.2)*

PS is specified as four layers with independent conformance. A venue may claim conformance per layer; layers evolve at different rates (the grammar is stable for years; parameters and runtimes are not).

Layer	Contents	Stability
PS/Core	Grammar, escaping, scopes, canonical references, round-trip property	High
PS/Capabilities	Entity discovery, capability documents, parameter registries, authorization surface	Medium
PS/Execution	Fulfillment policies, budgets, fallback chains	Medium
PS/Trace	Trace schema, tiers, fill reports, failure codes	Medium-high

A conformance claim names layers and version: e.g., `PS/Core 0.2 + PS/Trace 0.2 (user tier)`.

5. Directive scopes ✓

Scope	Syntax	Applies to	Example
Point	`@` entity reference; `/` action	The whole message (routing) or the reference site (context)	`@opus(temperature: 0.2) Summarize this. /concise`
Span	Reserved tag `<ps …>…</ps>`	The enclosed text region	`<ps @file:glossary.md>use these terms</ps>`
Document	Frontmatter fence `---ps … ---`	The entire prompt; the strict form	see §11

5.1 Span tag forms ✓

The `ps` tag accepts the same reference grammar as point scope (principle 7): `<ps @ref …>` casual; `<ps key="value">` canonical/XML-friendly (compiler-emitted).

5.2 v0.1/v0.2 span restriction ✓ *(v0.2 — was open; settled per external review)*

Spans are restricted to: **context inclusion, local instruction scope, tool availability, visibility/provenance marking**. Multi-model span execution (`<ps @haiku>…</ps>` routing a region to a different model) is **deferred to a separate orchestration extension**: it requires execution-order, data-flow, context-isolation, token-accounting, failure- and injection-boundary semantics that do not belong in the core language. The syntax remains forward-compatible; venues MUST reject (not ignore) model references in spans with `SPAN_EXECUTION_UNSUPPORTED`.

6. Entity references and resolution

6.1 Reference syntax ✓

```
reference   ::= "@" [ namespace ":" ] name [ "!" ] [ arglist ]
namespace   ::= "model" | "agent" | "plugin" | "tool" | "skill" | "file" | "ctx" | vendor-ext
name        ::= identifier ( ("." | "/") identifier )* [ "@" version ]
arglist     ::= "(" arg ( "," arg )* ")"
arg         ::= key ":" value                          (* JSON5-style scalars *)
action      ::= "/" name [ arglist ]
```

6.2 Canonical identifiers ✓ *(v0.2)*

Casual aliases (`@opus`, `@fast`) are legal input. Resolution MUST bind every reference to a **globally canonical, immutable, versioned identifier**, recorded in the canonical form and the trace:

```
@opus                  →  @model:anthropic/claude-opus-4-5@2026-05-01
@tool:search-contracts →  @tool:workspace/acme/search-contracts@3
@file:q3-report.md     →  @file:workspace/acme/9fc2…
```

Model aliases drift; dated snapshots do not. The trace never contains an unpinned binding.

6.3 Ambiguity and namespace resolution ✓

1. A bare name matching exactly one entity in the capability envelope binds to it.
2. Multiple matches are an **ambiguity condition**: implementations MUST surface it and MUST NOT silently guess (`ENTITY_AMBIGUOUS`).
3. Every binding, including the deciding rule, is recorded in the trace's resolution report.

? *Open: default namespace search order (proposal: session bindings → project config → `model` → `agent` → `plugin` → `tool` → `skill` → `file`).*

6.4 Environment-relative parsing — documented limitation ✓ *(v0.2)*

Because bare-name recognition depends on installed entities, identical text may parse differently across venues. PS guarantees a **visible** parse, not an identical parse. Normative mitigations: island highlighting is MUST for interactive implementations (§13); **consequential actions** (tool execution, connector access, data egress) MUST be either fully qualified or explicitly confirmed by the user; venues SHOULD offer a strict mode in which only `<ps>` / `---ps` content is executable.

7. Capabilities and parameters *(PS/Capabilities)*

7.1 Capability document ✓ *(v0.2)*

A conformant venue publishes a machine-readable capability document; implementations negotiate against it before fill:

```
ps_version: "0.2"
supports:
  scopes: [point, span, document]
  fulfillment: [strict, partial, best-effort]
  budgets: [estimated_cost, observed_cost, wall_time, first_token]
  cancellation: preferred          # required | preferred | unsupported
  trace_tiers: [user, developer]
namespaces:
  model: { discovery: "…", pinning: date-snapshot }
  tool:  { discovery: "…" }
```

A prompt MAY declare the PS version it targets (`ps: "0.2"` in frontmatter); venues MUST reject versions they cannot honor rather than degrade silently.

7.2 Parameters: normalization, not passthrough ✓ *(v0.2 — replaces v0.1 passthrough)*

There is no stable universal parameter registry. PS defines a small **normalized core** (`temperature`, `top_p`, `max_tokens`, `stop`, `seed`, `effort`) plus vendor-native names. Every parameter's journey is recorded as a **normalization record** in the fill report:

```
{ "requested":  { "effort": "max" },
  "normalized": { "reasoning.effort": "xhigh" },
  "applied":    { "reasoning.effort": "xhigh" },
  "status":     "translated",
  "mapping_rule": "vendor-profile/openai-2026-07" }
```

Status is one of `applied | translated | clamped | unsupported | ignored` — each with a defined fill consequence under the directive's policy. Undocumented passthrough is non-conformant: it creates the appearance of control without semantics.

8. Fulfillment *(PS/Execution)*

8.1 Policies ✓ *(settled; external review concurs)*

Inspired by exchange order types: a routing directive is an order; the venue fills it or reports why not.

Policy	Trading analog	Semantics
`strict`	Fill-or-Kill	Every requirement satisfied exactly, or the directive fails pre-execution. No silent substitution.
`partial`	Immediate-or-Cancel	Fill what can be filled; every drop recorded.
`best-effort`	Market order	Venue discretion; all degradations still recorded.

Default (✓): split — `strict` on entity binding (`@opus` means opus, never a silent substitute), `partial` on parameters (unsupported values drop with a fill-report entry). **`!` semantics (✓ v0.2.1 — clarified per external review):** since entity binding is already strict by default, `!` escalates the **entire directive** to strict: every parameter becomes a hard requirement (no drop, no clamp; translation only where the vendor profile declares it semantics-preserving), and any degradation is a fill failure instead of a reported drop. `@opus` = strict entity, partial params; `@opus!` = strict everything. Named forms: `fill="strict"` (span/document), `fill: strict` (frontmatter; document level = all-or-none). **?** *Open (severity/consent): degradation-severity classes and an optional `confirm-on-degrade` preflight mode for consequential settings — a post-hoc report is not consent.*

8.2 Failure codes ✓ *(v0.2)*

Fill failures and degradations carry standardized codes (extensible registry, `PS_`-free core set):

```
ENTITY_NOT_FOUND         ENTITY_AMBIGUOUS          ENTITY_UNAUTHORIZED
PARAM_UNSUPPORTED        PARAM_CLAMPED             PARAM_IGNORED
BUDGET_PREFLIGHT_REJECTED  BUDGET_EXCEEDED         TIMEOUT_NOT_CANCELLED
DATA_POLICY_BLOCKED      SPAN_EXECUTION_UNSUPPORTED  VERSION_UNSUPPORTED
FALLBACK_EXHAUSTED
```

9. Budgets and fallback chains *(PS/Execution)*

9.1 Budgets ✓ *(v0.2 — replaces generic `limit()`)*

`limit(…)` remains as syntax sugar, expanding to a **budget** with defined measurement points:

```
budget:
  estimated_cost_max: $0.10     # pre-flight: reject if floor estimate exceeds
  observed_cost_max:  $0.15     # runtime: abort (if cancellable) when exceeded
  wall_time_max:      5s        # submission → completion, tool calls included
  first_token_max:    1s        # streaming responsiveness bound
  cancellation:       preferred # what the directive requires: required | preferred
  scope:              attempt   # attempt | chain
```

Semantics: a violated budget is a fill failure. Venues declare their enforcement capability in the capability document (§7.1); a directive requiring `cancellation: required` fails pre-flight (`BUDGET_PREFLIGHT_REJECTED`) against a venue declaring `unsupported`. A timeout the venue could not cancel is reported as `TIMEOUT_NOT_CANCELLED` with observed cost — enforcement is honest-by-declaration, never pretended. Implementations MUST NOT silently degrade output (e.g., truncate) to satisfy a budget. `scope: chain` bounds the **total** across fallback attempts *(resolves v0.1 open question)*.

9.2 Fallback chains ✓ *(structure settled; authorization rule added in v0.2)*

PS has **no general conditionals** — no predicates over content, no branching on output (delegated to orchestration frameworks). The only re-routing trigger is a **fill failure**. Precedent: CSS font stacks; trading's contingent orders.

```
route    ::= step ( "else" step )* [ "else" terminal ]
step     ::= reference [ limitclause ]
terminal ::= "fail" | "ask"
```

```
@opus!(temperature: 0.2) limit(wall_time: 5s, estimated_cost: $0.10)
  else @model:openai/gpt-5.6@2026-07-01 limit(wall_time: 3s)
  else ask
```

First successful fill wins; the fill report records every attempt with outcome, codes, and measured cost/latency. **Authorization rule (✓ v0.2):** every step is checked against the same capability envelope — data classification, consent, allowlists, residency, retention, connector authorization. A failed primary route never makes a prohibited secondary route permissible; a cross-provider fallback is a data-governance decision, not merely an availability decision, and blocked steps report `DATA_POLICY_BLOCKED`.

10. Authority and precedence ✓ *(v0.2 — replaces v0.1 single chain; critical fix)*

Precedence has **two independent dimensions**. The v0.1 single chain (`system < … < span`) was unsafe: read literally, user content could override platform policy.

10.1 Authority precedence (who may decide)

```
platform policy  >  organization/workspace policy  >  application/developer policy
                 >  user policy  >  prompt content
```

Lower authority may **narrow** permissions, budgets, and entity sets; it may never expand them (principle 8).

10.2 Specificity precedence (which value wins, within one authority level)

```
document  <  point  <  span        (inner/more-specific wins)
```

10.3 Normative rule

> A more specific directive overrides configuration **only within the capability envelope granted by higher-authority policy.** An out-of-envelope directive is a fill failure (`ENTITY_UNAUTHORIZED` / `DATA_POLICY_BLOCKED`), never a silent ignore — the user always learns that authority, not availability, was the reason.

Every envelope constraint that suppressed a directive is recorded in the trace at the requesting user's tier (as a typed refusal, not necessarily with the policy's content).

10.5 Refusal reasons and verified attributes 🔹 *(v0.2.1)*

A directive can be blocked for categorically different reasons that current interfaces collapse into one undifferentiated outcome (e.g., a single "switched model" banner covering safety blocks, capacity substitution, and org-policy denials alike). PS requires the reason be **typed** in the trace, distinguishing at minimum: availability (entity unavailable/capacity), **safety** (`SAFETY_BLOCKED`, with a category and the provenance of the *triggering segment* — which may be a retrieved document or tool result, not the user's message), org/data policy (`DATA_POLICY_BLOCKED`), and authorization (`ENTITY_UNAUTHORIZED`). The receipt names the reason, the deciding authority, and — normatively — an available **recourse path** where one exists (appeal, verification program, feedback channel).

Verified principal attributes (the credential-informed envelope). A safety or policy envelope MAY be widened for a principal presenting a *verified attribute* (an identity proof, a credential such as a cyber-verification-program membership, an institutional affiliation). This is expressed entirely within the authority model and is **not** non-escalation-violating: the wider envelope is granted by a *higher* authority (the verifier/provider reading the attribute), never claimed by prompt content. The signed attribute is an **input to** the authority-holder's decision, never a self-granted capability; the decision and the attributes it relied on (by reference, not by disclosing the credential) are recorded. Verified-attribute policies MUST NOT be mandated by PS — identity-gating carries chilling, privacy, and gatekeeping costs, and whether to use the mechanism is contestable policy left to the authority. *(Existing practice: providers already run verification programs that widen safeguards for verified defensive-security researchers — PS standardizes the seam, not the program.)*

? *Open (companion-paper territory): appeal-loop semantics; credential formats and privacy-preserving attribute proofs; interaction of verified envelopes with the oversight tier.*

10.4 Policy system integration (PEP/PDP) 🔹 *(v0.2.1)*

PS does not define a policy language, and it adds **no new policy decisions** — the organization keeps its existing engine. What PS adds is what the existing control stack cannot produce: (1) **prompt-level decision semantics** — network- and key-layer controls are blind inside the request; PS's decision requests carry the entities, classifications, and fallback routes a meaningful policy needs to see; and (2) **per-request enforcement evidence** — today an admin setting is a request to the vendor with no proof it fired; typed refusals and fill reports make every policy outcome verifiable. PS defines the **enforcement seam**: the PS runtime is a *policy enforcement point* (PEP); the organization's policy engine is the *policy decision point* (PDP), attached at the org/workspace authority level. At resolution time, for each directive, the PEP submits a decision request — **principal** (identity/groups from the IdP), **entity reference** (canonical identifier), **context labels** (data classification of referenced files/sources), **jurisdiction/residency** — and the PDP answers: in-envelope or refused (with code), applicable budget, permitted egress set for fallback steps. Decisions and refusals are recorded in the trace (policy hits, not policy content). **Bindings** to concrete engines (OPA/Rego, Cedar, XACML-style) are external versioned documents, declared in the capability document.

Worked example (🔹 v0.2.1 — illustrative; decision-interface schema remains OQ#10). User input: `@opus! Summarize @file:q3-acquisition-memo.pdf else @gemini-flash`. At resolution time — before assembly, before any call — the PEP submits per-binding decision requests to the org's existing engine:

```
{ "principal": { "id": "user:meas@acme", "groups": ["finance"] },
  "action": "route",
  "route": [
    { "step": 1, "entity": "model:anthropic/claude-opus-4-5@2026-05-01",
      "provider_jurisdiction": "EU" },
    { "step": 2, "entity": "model:google/gemini-flash@2026-06-01",
      "provider_jurisdiction": "US", "role": "fallback" } ],
  "context": [ { "entity": "file:acme/9fc2…", "classification": "confidential" } ],
  "budget_scope": "attempt" }
```

Against an unchanged org policy (e.g., Rego/Cedar: *confidential sources route only to EU-approved models*), the PDP returns per-step decisions: step 1 `permit`, step 2 `deny(egress)`. The PEP fills step 1 and marks step 2 `DATA_POLICY_BLOCKED` **before any failure could trigger it** — the chain is judged as one decision, not discovered call-by-call — and the trace records the `policy_decision_id`s.

Why a call-level gateway PDP is not equivalent. Gateways with policy sidecars can and do filter API calls today. But a call-level decision point sees flattened text and a model name; it cannot see (1) **typed references before flattening** — by the time the gateway inspects the call, the confidential file is extracted text and its classification label died in assembly, whereas PS decides on the reference while the label exists; (2) **the chain** — the fallback call does not exist yet at gateway time, and when it fires it is a fresh call that may

individually pass; (3) **injected expansion** — which skill added what, or prompt-content attempts at tool-access escalation (inert-content rule, §13); (4) **its own effectiveness** — a gateway drop is silent; a PS refusal is typed, user-visible, and receipted. The layers compose: gateways remain useful as defense in depth below the PS seam.

Note — the externalization pattern: vendor profiles (§7.2), oversight profiles (§12.3), and policy bindings (§10.4) are the same architectural move: the core spec defines interface + semantics; stakeholder-specific mappings live outside and version independently. Enterprise-relevant consequences already in core: typed refusals as policy audit events; org-authority budgets; the fallback-authorization/egress rule (§9.2); pinned identifiers as model change management (§6.2); capability documents as procurement artifacts (§7.1).

11. Canonical form and the strict document ✓

Canonical form: frontmatter (all routing/config, canonical pinned identifiers, defaults materialized, PS version declared) + body (prose with islands replaced by resolved references).

```
---ps
version: "0.2"
route:
  - { model: "model:anthropic/claude-opus-4-5@2026-05-01",
      params: { temperature: 0.2 }, fill: strict,
      budget: { wall_time_max: 5s, estimated_cost_max: $0.10 } }
  - { model: "model:openai/gpt-5.6@2026-07-01" }
skills: [ "skill:workspace/acme/concise@1" ]
context: [ "file:workspace/acme/9fc2…" ]
---
Summarize the report.
```

Round-trip property (normative): `compile(casual) ≡ compile(strict(compile(casual)))`. The intent-reification layer MUST output this strict form, editable and re-submittable.

12. The Prompt Trace *(PS/Trace)*

12.1 Structure ✓ *(settled; normative schema: `prompt-trace.schema.json`)*

The trace is a **turn-level object** with an ordered `events` array interleaving *inference records* and *boundary events*. Three settled decisions:

- **Shape (D1): flat segments + provenance steps.** Ordered segment list whose concatenation reproduces the compiled prompt byte-exactly within tier scope (rule R1); each injected segment carries a pointer into a step table whose parent chains reconstruct the expansion tree (R2, R3). Source-map / W3C-PROV pattern.
- **Turn scope (D2): tiered, with the boundary rule.** Routing record + fill report MUST for every venue invocation; full segments MUST for user-initiated inferences, SHOULD for internal continuations (absence *declared* via `coverage`, never silent — R6). Everything crossing the venue boundary (tool I/O) MUST be recorded; tool internals are explicitly outside the perimeter.
- **Content (D3): hybrid three-state union.** `inline` (≤ threshold; floor 4 KB for the user tier, R4) | `external` (sha256 + length + handle; dedups across loop steps) | `withheld` (salted digest + length band + minimum-visible tier). Oversight-tier exports MUST support **materialized mode** — external content resolved and embedded at export time, with blob retention governed at oversight scope (R5).

MUST: compiled prompt (tier-scoped); resolution report (every binding + deciding rule, ambiguities, envelope refusals); routing record incl. **fill report** (§8.2 codes, normalization records, per-attempt outcomes with measured cost/latency). **SHOULD:** dry-run (pre-execution) trace; stable segment IDs for diffing. **MAY:** any presentation.

12.2 Visibility tiers ✓ *(v0.2 — replaces flat redaction)*

Tier	Audience	Contents
User	End user	User-visible sources, selected tools, applied parameters, typed envelope refusals, fill report
Developer	Application owner	+ application instructions, tool definitions, runtime events
Operator	Platform personnel	+ infrastructure and policy diagnostics
Oversight	Auditors & regulators (via profiles, §12.3)	Evidence-grade export per the applicable oversight profile

Completeness is **tier-relative**: a tier's trace is complete with respect to content at or below that tier's authority; higher-tier content appears as a **typed, sized placeholder** (`{type: system_policy, tier: operator}`). **?** *Open ("the transparency floor"): what placeholder detail is safe — exact lengths and content hashes can leak (dictionary attacks against candidate texts); banding and salted digests are candidate mitigations. This is a negotiation between auditability and confidentiality, flagged as a research question.* Trace availability interacts with venue data-retention configuration; the capability document declares supported tiers.

12.3 Oversight: evidence substrate and profiles ✓ *(v0.2.1)*

Enterprise audit and government/regulatory oversight are **distinct lenses**. An auditor's access is granted by contract *inside* the authority ladder; a regulator's access is compelled by law from **above platform policy** (§10 note below). Regulatory requirements are jurisdictional, sectoral, and mutually conflicting (e.g., EU AI Act Art. 12 automatic record-keeping vs. GDPR minimization vs. WORM-retention financial rules), and they demand an evidentiary standard — tamper-evidence, authenticity, chain of custody (contributing to, not constituting, legal non-repudiation) — that internal audit does not.

PS therefore specifies no jurisdiction's rules. Instead:

- **Evidence substrate (PS/Trace core).** Properties any oversight regime composes from: integrity chaining or signing of trace records (SHOULD in core; oversight profiles are expected to make these MUST, with canonical serialization, key custody, and verification procedures profile-specified); completeness attestations (what coverage the venue claims, signed); **signed sequence numbers with explicit gap records** (an omitted event leaves a visible hole, not silence); trusted timestamps (SHOULD); retention and residency metadata fields (MUST when the oversight tier is supported); **provable redaction** — a withheld segment's existence, kind, and timing are demonstrable without revealing content; the fully **materialized export** (§ Decision 3) with blob retention governed at oversight scope. Because a venue-produced trace is self-asserted, profiles MAY additionally require independent trust mechanisms: append-only transparency logs, cross-reconciliation against gateway or billing records, independently operated collectors, or remote attestation.
- **Oversight profiles (external documents).** Jurisdiction- or framework-specific mappings (e.g., an EU-AI-Act profile, a SOC 2 profile) declaring which substrate properties are MUST and with what parameters (retention period, residency, signature regime). Profiles are versioned and maintained outside the core spec — the PDF/A / FHIR national-profile / OTel semantic-convention pattern. The capability document declares supported profiles.

Authority note (amends §10.1): the authority ladder is crowned by an acknowledged but externally governed level — *law and regulation* — above platform policy. Non-escalation continues to bind prompt content; disclosure obligations, however, may be compelled from this level, making the transparency floor (§12.2) **authority-relative**: content a venue may withhold at the user tier can be compellable at the oversight tier under an applicable profile.

13. Content safety and input marking ✓

- **Provenance-typed parsing (inert content) ✓ *(v0.2.1 — security-critical, from external review)*:** Only explicitly designated **authoring segments** — text the principal typed or an authority-checked surface produced as PS — are parsed for directives. Retrieved documents, file contents, tool results, quoted messages, and model-generated text are typed **inert**: PS-shaped text inside them (e.g., a document containing `@tool:send_email!(…)`) is content, never a directive, and cannot introduce directives except through an explicit, authority-checked *promotion* operation recorded in the trace. This rule is prior to and stronger than fence exemption; without it, PS would be a prompt-injection and capability-escalation surface.
- **The parameterization principle ✓ *(v0.2.1)*:** security rests on provenance, never on escaping. PS is parsed only at the interface layer, only over authored segments — models never execute PS, so inert content is never "escaped for safety"; it is never parsed at all. Content inclusion is **by reference** (canonical id + digest), the discipline that ended SQL injection: data is never concatenated into the command channel. Escapes (`\@`, `\/`, fences) exist solely as authoring ergonomics for deliberate literals; no security property may depend on them.
- **Inner content and nesting ✓ *(v0.2.1)*:** a span's inner prose is authored text — islands within it parse normally; content *resolved into* a span via its references remains inert. Canonical-form serialization MUST carry content segments by reference or in typed data fields, never spliced into directive text, so serialize-then-recompile cannot promote inert content (round-trip safety). Nested-span conflict semantics: pending conflict table (OQ). **Model-generated PS** (including the reification layer's proposed strict form) is inert until explicitly accepted by the user — reification acceptance is the archetypal authority-checked promotion, and the promotion event is recorded in the trace.
- Only the `ps` tag namespace is live; all other `<…>` is content.
- `@/ /` islands require a word boundary and a resolvable or qualified name; emails and paths do not parse.
- Escapes `\@`, `\/` and fenced code blocks are always content.
- **Island highlighting is MUST for interactive implementations** *(v0.2 — upgraded from SHOULD per review)*: every recognized directive is visually marked before submission. Non-interactive (API) callers use dry-run traces for the same guarantee.

13.1 Internationalization ◈ *(v0.2.1)*

PS prompts are natural language in any script; the grammar must not assume Latin text.

- **Sigils are script-neutral, with width equivalence.** `@` and `/` are language-independent and present on keyboards worldwide; parsers MUST treat full-width forms (U+FF20 `@`, U+FF0F `／`), commonly produced by CJK input methods, as equivalent to their ASCII forms.
- **Unicode identifiers.** Entity names follow Unicode identifier rules (UAX #31) with NFC normalization — `@file:របាយការណ៍.md` is valid. Namespace keywords, policy names, and `else` remain English in the **canonical form** (the portability lesson of Excel's localized function names: localized *canonical* syntax destroys portability). Implementations MAY accept localized input aliases for keywords and units; these compile to canonical form — internationalization rides the same casual→strict machinery as everything else, and the round-trip property applies.
- **Script-aware boundaries.** The "word boundary before a sigil" rule is defined by Unicode segmentation (UAX #29), not ASCII `\b`

— scripts without inter-word spaces (Khmer, Thai, Japanese, Chinese) MUST still get correct island detection. Conformance test corpora MUST include non-spacing-script cases.

- **Bidirectional text safety.** In RTL contexts (Arabic, Hebrew), editors SHOULD wrap islands in bidi isolates (FSI…PDI) for stable rendering. Security-normative: parsers MUST reject or neutralize bidi control characters *inside* directive islands — Trojan Source-style reordering (CVE-2021-42574) could otherwise visually disguise what a directive does, defeating mandatory highlighting.
- **Confusables.** Homoglyph references (`@opus` with Cyrillic `о`) are a spoofing vector against entity binding. Resolvers SHOULD apply confusable detection (UTS #39) and surface skeleton-matches as ambiguity conditions rather than silently binding.
- **Localized scalars.** Budget/parameter input MAY use locale conventions (decimal comma, local currency); canonical form and traces always use canonical units (USD, `ms` / `s`, dot-decimal).

? *Open: governance of localized keyword-alias tables (per-language registries? vendor-defined?); which locales the conformance suite must cover.*

14. Determinism taxonomy ✓ *(v0.2)*

Documentation and conformance language MUST distinguish, and never conflate:

Kind	PS guarantees it?
Routing-contract determinism — parsing, canonicalization, resolution, and policy evaluation under a pinned environment; execution choices recorded	**Yes** (within envelope; this is PS's claim — renamed from "execution determinism" per external review: tool calls, races, and provider behavior are not deterministic and are *recorded*, not guaranteed)
Generation determinism — identical output text	**No** (even with `seed`; vendor implementation varies)
Semantic reproducibility — same meaning across runs/venues	**No** (retrieval, web content, tools, policies drift)

`@model:x(seed: 42)!` guarantees the seed was applied — not that the output is identical.

15. Out of scope (delegated) ✓

Output/decoding constraints (LMQL, Guidance) — params/normalization is the integration point. Agent orchestration graphs (DSPy, LangGraph) and multi-model span execution — orchestration extension. Content-conditional branching — orchestration frameworks.

16. Open questions queue

1. Default namespace search order (§6.3).
2. ~~Trace JSON schema decisions~~ **Resolved** — flat+provenance / tiered turn scope / hybrid content confirmed; normative schema published (§12.1).
3. The transparency floor: placeholder detail vs. leakage (§12.2).
4. `else ask` terminal: interaction contract with intent reification.
5. Capability-document schema detail (§7.1) and vendor-profile registry governance.
6. Failure-code registry governance (who mints codes; vendor extensions).
7. Vendor mapping table (Claude, ChatGPT, Cursor, Copilot, Gemini) — separate document.
8. Orchestration extension scoping (multi-model spans, §5.2).
9. Oversight-profile governance: who authors and maintains jurisdiction profiles (standards body? consortium?); how profile conformance is asserted and verified (§12.3).
10. PDP decision-interface schema (§10.4): request/response shape, latency budget for per-directive policy evaluation, caching semantics, and binding governance.
11. *(from external auditor-lens review, 2026-07-19)* Identifier governance: who controls canonical-identifier uniqueness, vendor prefixes, deprecation, alias ownership, disputes (§6.2).
12. Budget measurement profiles: tariff/price-sheet version identifiers, cached-token treatment, queueing inclusion, cancellation-attempt evidence (§9.1).
13. Trace-privacy deployment profiles: field-level encryption, access logging, deletion, legal hold, retention conflicts, purpose limitation (§12.2).
14. Version-baseline consolidation: merge v0.2.1 markers into a clean v0.3 baseline before public spec release; conformance tests reference exactly one baseline.
15. Conformance vs. certification: negative/adversarial test requirements, signed capability documents, and a path to independent verification (currently: two author-controlled implementations demonstrate spec precision, not third-party assurance).

---

*Naming: properties are routing **control** and routing **transparency**; the artifact is the **Prompt Trace**; fulfillment vocabulary is `strict` / `partial` / `best-effort`.*